Cryptographie R309

29 septembre 2025

Chiffrement monoalphabétique, Partie I

Objectif Le but de cette séance consiste à présenter une généralisation du chiffre de César : Le chiffrement par substitution monoalphabétique. Comme lors de la séance précédente, on implémentera un algorithme de chiffrement et de déchiffrement. Dans un second temps, on proposera une méthode de décryptage basée sur une méthode d'analyse fréquentielle.

Rendu `mono_alphabetique.py`

Le code devra être réaliser sous `Python`, les fonctions demandées devront être sauvegardées dans un fichier nommé `mono_alphabetique.py`. Le fichier devra être versé sur le Moodle dans la boîte de dépôt prévue. Attention, le fichier rendu ne doit pas contenir de variables globales ni même de tests. Il en doit contenir que les fonctions demandées (en rose dans l'énoncé).

Test `test-mono_alphabetique.py`

Un script de test est fourni afin que chaque étudiant puisse valider son code au fur et à mesure du TP.

Contexte Bob et Alice réalisent que, même en changeant régulièrement de clé, le chiffre de César n'assure pas de protection suffisante, notamment contre l'approche "brute force" consistant à tester les 26 clés possibles. Plutôt que d'utiliser une substitution circulaire, ils optent pour n'importe quelle permutation de l'alphabet.

Le chiffrement par substitution Une clé est cette fois définie par une permutation de l'alphabet

$$\mathcal{A} = \{A, B, \cdots, Y, Z\},$$

c'est à dire une bijection de $\mathcal{A}$ vers $\mathcal{A}$. On note $\mathcal{S}$ l'ensemble des permutations. Bob et Alice décident d'utiliser la clé σ définie par

```
SMUGCDABWNVEJXPIHTRFQLZYKO
```

Listing 1 – Clé définie par une chaîne de caractères

qui correspond à la permutation de l'alphabet

$$\sigma(A) = S;\ \sigma(B) = M;\ \sigma(C) = U;\ \cdots \tag{1}$$

Bob souhaite envoyer à Alice le message

`Ils ne nous auront pas cette fois.`

Comme d'habitude, il supprime les accents des lettres accentuées, transforme les lettres minuscules en majuscules puis élimine tout symbole n'appartenant pas à l'alphabet. Son message après ce pré-traitement devient donc

`ILSNENOUSAURONTPASCETTEFOIS`

Ensuite il remplace les lettres du message à l'aide de la permutation définie par la clé choisie. À cet effet, il lui suffit de superposer l'alphabet et la clé pour obtenir les correspondance entre chaque lettres.

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ
SMUGCDABWNVEJXPIHTRFQLZYKO
```

Il doit donc remplacer le lettre `A` par la lettre `S`, la lettre `B` par `M`, etc. Le message une fois codé, devient donc

`WERXCXPQRSQTPXFISRUCFFCDPWR`

Pour le décrypter, il suffit à Alice d'appliquer l'opération réciproque, c'est à dire de remplacer la lettre `S` par la lettre `A` et ainsi de suite. Ceci revient à appliquer la clé "inverse" de celle utilisée par Bob, c'est à dire

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ
GHEFLTDQPMYVBJZOUSARCKINXW
```

Implémentation On se propose d'implémenter une procédure de chiffrement pouvant être utlisée par Alice et Bob.

1. *(Prétraitement du message à chiffrer)* L'alphabet utilisé est défini comme une liste `Python`

```
Alphabet=[chr(k) for k in range(65,65+26)] #Lettres majuscules {A,...Z}
```

   a) Écrire une fonction `preprocess_message` qui prend en argument une chaîne de caractère et renvoie cette même chaîne après en avoir supprimé les accents des lettres accentuées et transformé toutes les lettres minuscules en majuscules. À cet effet, on pourra éventuellement utiliser la libraire `unidecode` de `Python`.

   b) Écrire une fonction `clean_message` qui prend en arguments un message et l'alphabet et supprime tout caractère non alphabétique (i.e. n'appartenant pas à l'alphabet passé en argument).

2. *(Conversion clé / chaîne de caractères)* Si représenter la clé de codage par une chaîne de caractères permet une définition et affichage compacts, ce n'est pas vraiment pratique du point de vue de l'implémentation. L'usage d'un dictionnaire `Python` est en effet plus adapté.
   Écrire les fonctions
   - `string_2_key` qui prend en argument une chaîne de caractères et un alphabet et renvoie un dictionnaire `Python` correspondant à la clé définie par la chaîne de caractère, comme la clé de l'équation (1) définie par la chaîne du Listing 1.
   - `key_2_string` qui prend en argument un dictionnaire représentant une clé et un alphabet et renvoie la chaîne de caractère représentant la clé.

3. *(Chiffrement monoalpabétique)* Écrire une fonction `substitution` qui prend en argument un message correctement formaté et une clé et retourne le message codé.

4. *(Déchiffrement)* Écrire une fonction `inverse_key` qui à partir d'une clé détermine sa clé inverse permettant le décodage. La réponse d'Alice à Bob est

   ```
   WEDSQGTSWFHQPXUBSXACGCUECTCAQEWCTCJCXFNCUTPWR
   SLPWTFTPQLCTQXJPKCXGCACXCTCTGCRUECRSECSFPWTCR
   ```

   Bob se dit que, définitivement, Alice est géniale. Quel est son message ?

5. *(Génération de clés)* Afin de générer chaque jour une nouvelle clé aléatoire, Alice a l'idée suivante : Dans un premier temps, on définit un vecteur v composé de 26 nombres aléatoires compris entre 0 et 1. Pour tout $i \in \{1, \cdots, 26\}$, une valeur $v_i \in (0, 1)$ est tirée de manière aléatoire et uniforme. Dans un second temps, les éléments du vecteurs sont triés par ordre croissants. C'est à dire qu'on détermine σ, permutation de $\{1, \cdots, 26\}$, telle que pour tout $i \in \{1, \cdots, 25\}$,

   $$v_{\sigma(i)} \leq v_{\sigma(i+1)}.$$

   Enfin, la permutation σ sur $\{1, \cdots, 26\}$ permet de définir une permutation sur l'alphabet en identifiant lettre de l'alphabet et son rang dans l'alphabet.
   Écrire une fonction `random_key` qui prend en argument un alphabet et génère automatiquement une clé aléatoire. Selon la procédure décrite précédemment.

Attention : La méthode `shuffle` de `Python` permet de générer automatiquement des permutations. On demande ici d'implémenter l'algorithme décrit (et non d'appliquer `shuffle`).

Analyse fréquentielle des lettres Quelques jours plus tard, Alice et Bob échangent le message suivant

```
GQTSXFESRCUPXGCAQCTTCJPXGWSECWENPQCQXTPECJSNCQTGSXRESUTKIFSXSEKRCGCES
JSUBWXCCXWAJSQFWEWRCCISTECRSTJCCRSEECJSXGCREWXLCXFWPXGCJSUBWXCRQRSXFG
CITPUCGCRCECUFTPXWHQCRECRMPJMCRDCTSISRRCTECGCUTKIFSACSIEQRWCQTRJWEEWC
TRGCJCRRSACRISTNPQT
```

Listing 2 – Echange entre Alice et Bob

Nous allons tenter de décrypter ce message sans en connaître la clé. À cet effet, nous allons effectuer une analyse sur la fréquence d'apparition des lettres, similaire à ce qui avait été utilisé pour casser le code César. Tout d'abord, nous introduisons une fonction J_1 qui mesure la plausibilité d'un message en clair en langue française. Il existe plusieurs choix possibles, mais nous allons conserver la même mesure que lors de la précédente séance.

$$J_1(m) = \sum_{x \in \mathcal{A}} |F_1(m,x) - \overline{F}_1(x)|.$$

Avec

- $F_1(m,x)$ la fréquence d'apparition de la lettre x dans le message m ;
- $\overline{F}_1(x)$ la fréquence d'apparition de x dans un corpus de référence.

Pour un message codé m, plutôt que de rechercher la clé, nous allons chercher la clé inverse, c'est à dire la permutation σ pour laquelle $\sigma(m)$ apparaît en clair. Dans ce cas, cela doit être un texte en français et la valeur de $J_1(\sigma(m))$ doit être relativement faible. On souhaite donc résoudre le problème

$$\min_{\sigma \in \mathcal{S}} J_1(\sigma(m))$$

où $\sigma(m)$ désigne le message m codé à l'aide de la clé σ.

6. Pourquoi n'est-il pas envisageable de tester toutes les clés possibles pour minimiser J_1 ?
   En supposant qu'un ordinateur puisse tester un milliard de clés par seconde, combien de temps lui faudrait il pour obtenir la solution ?

7. *(Calcul des fréquences)*

a) Définir une fonction `load_monogrammes` qui retourne le dictionnaire des fréquences d'apparition des lettres dans le corpus de référence. On utilisera le fichier `monogrammes_tries.txt` qui donne le nombre d'occurrences de chaque lettre de l'alphabet sur un corpus extrait de Wikipédia.
b) Définir une fonction `F` qui prend en arguments un message et un alphabet et renvoie les fréquences d'apparition des lettres de l'alphabet dans le message.

8. *(Évaluation de J_1)* Implémenter une fonction `J1` qui prend en argument un message m et les fréquences de référence en français en renvoie $J_1(m)$.

9. *(Minimisation de J_1)* L'approche brute force est en fait totalement inutile pour minimiser J_1. Une procédure très simple de tri permet d'obtenir la solution optimale. Quelle est-elle ? Quelle est sa complexité en fonction de la taille de l'alphabet ? Écrire une fonction `minJ1` qui prend en argument un message codé ainsi que les fréquences de référence et retourne la permutation minimisant J_1 obtenue par ce procédé de tri.

10. *(Test de l'algorithme de décryptage)* Nous allons tester l'algorithme de décryptage proposé sur un texte en clair issu de la littérature française fourni par `proust.txt`. Le message n'étant pas codé, on espère retrouver la permutation identité. Est-ce le cas ? Combien de lettres sont correctement trouvées ?

11. L'algorithme permet il de décrypter le dernier échange entre Alice et Bob ?